



University of Santo Tomas
College of Information and Computing Sciences
Department of Computer Science
A.Y. 2024 - 2025



Health Tracker App - App to Track your Fitness

APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES 3

(MOBILE PROGRAMMING) — ICS26011

3CSE



INVENTADO, Charles Fredric G.



RODELAS, John Vincent B.



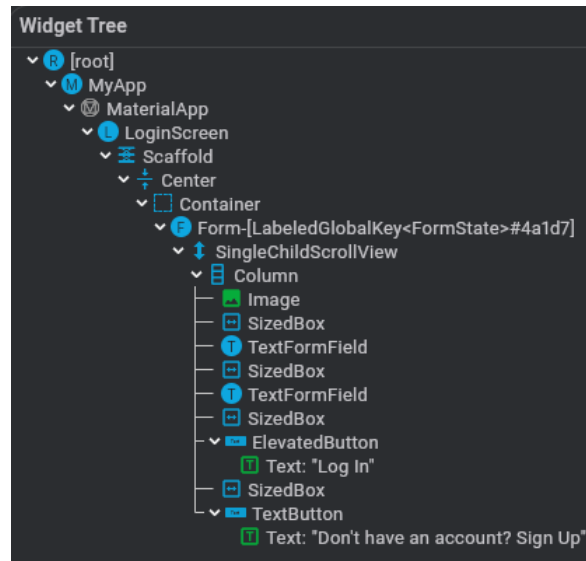
VALLES, James Vincent V.

December 17, 2024

Table of Contents

Table of Contents.....	1
Widget Tree.....	2
Flutter Project Code.....	9
App Demo.....	13

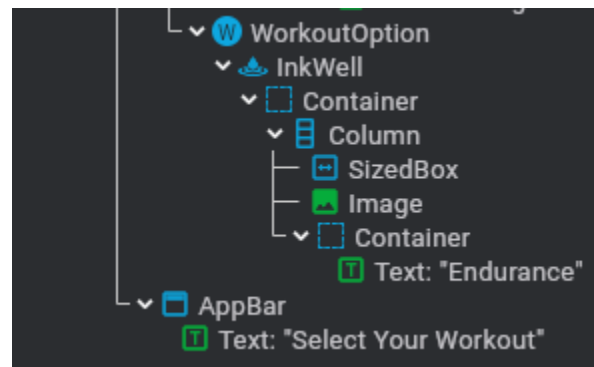
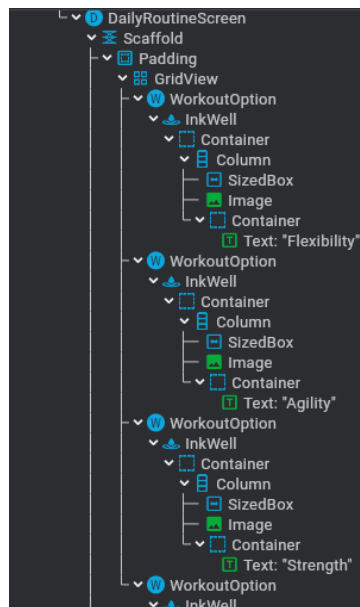
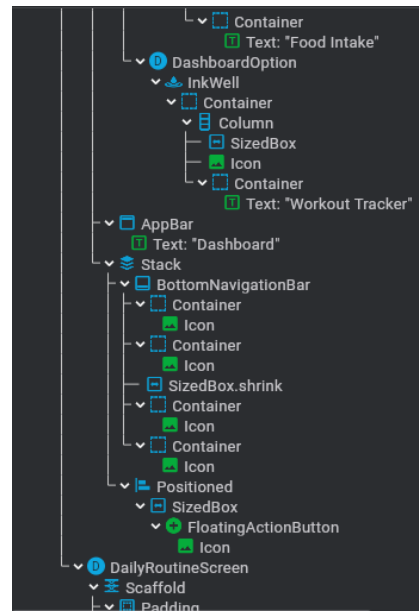
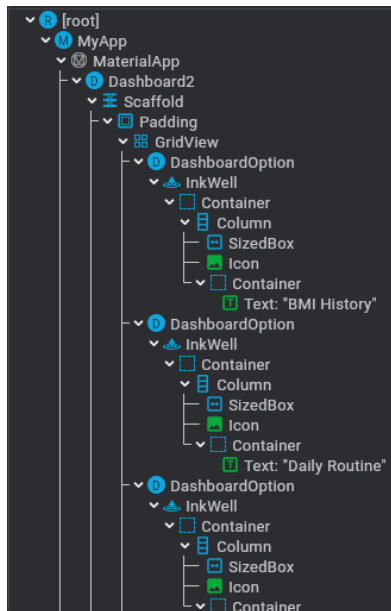
Widget Tree



1.0 Login and Register Widget Tree Provided by the Flutter Tools



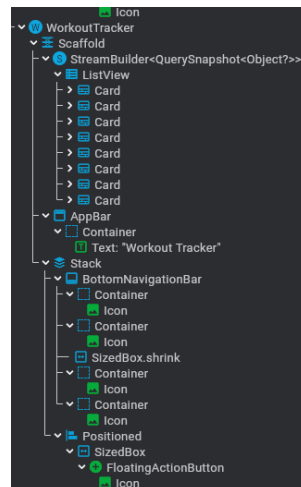
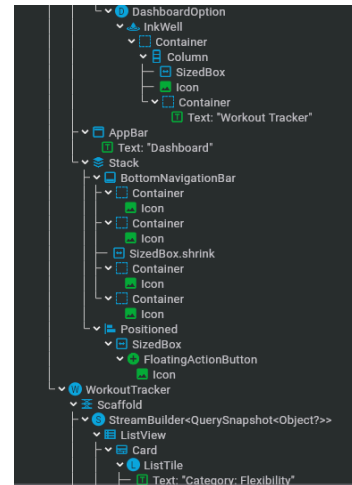
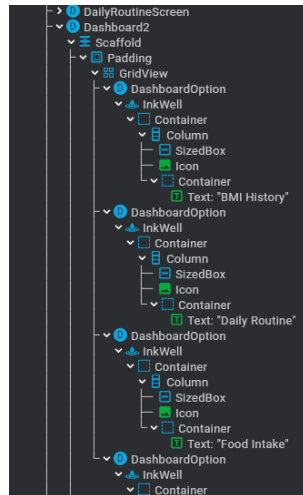
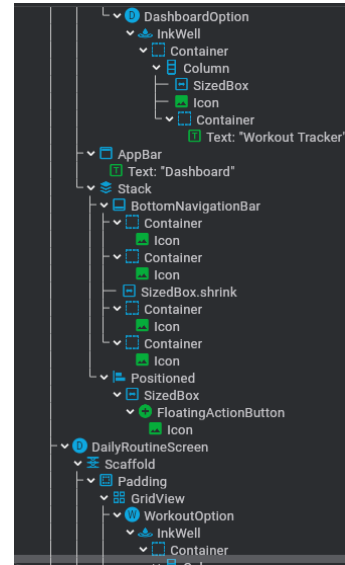
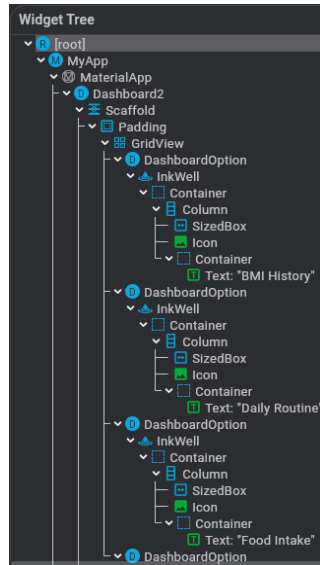
1.1 Dashboard Widget Tree Provided by the Flutter Tools



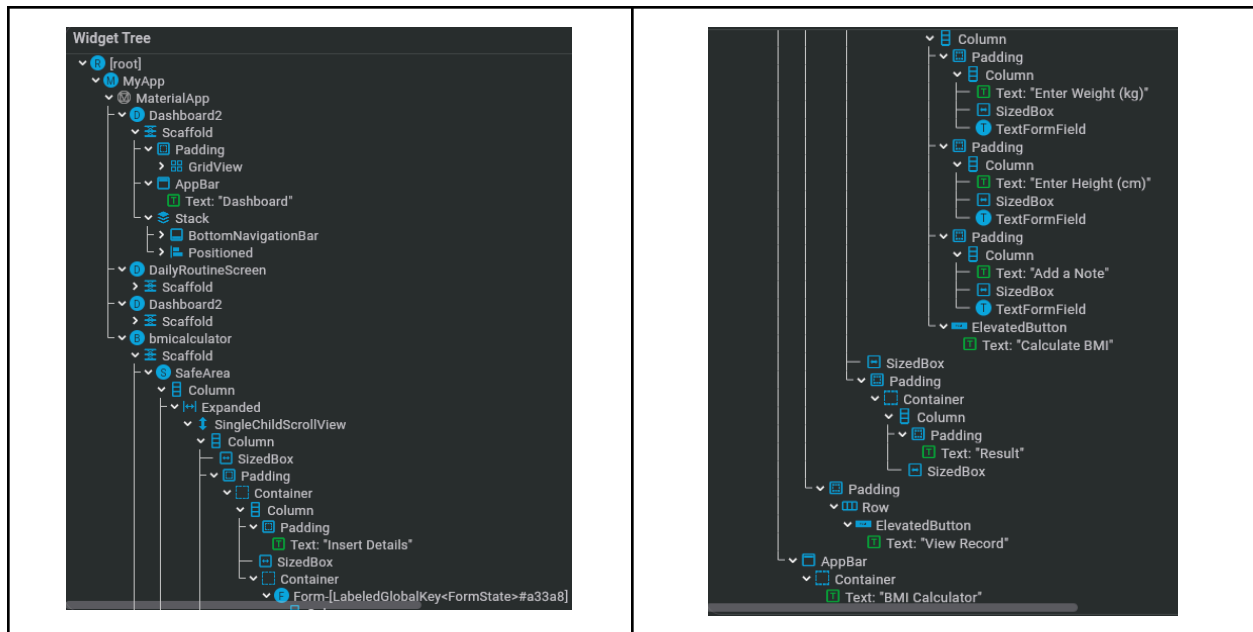
1.2.1 Daily Routine Widget Tree Provided by the Flutter Tools



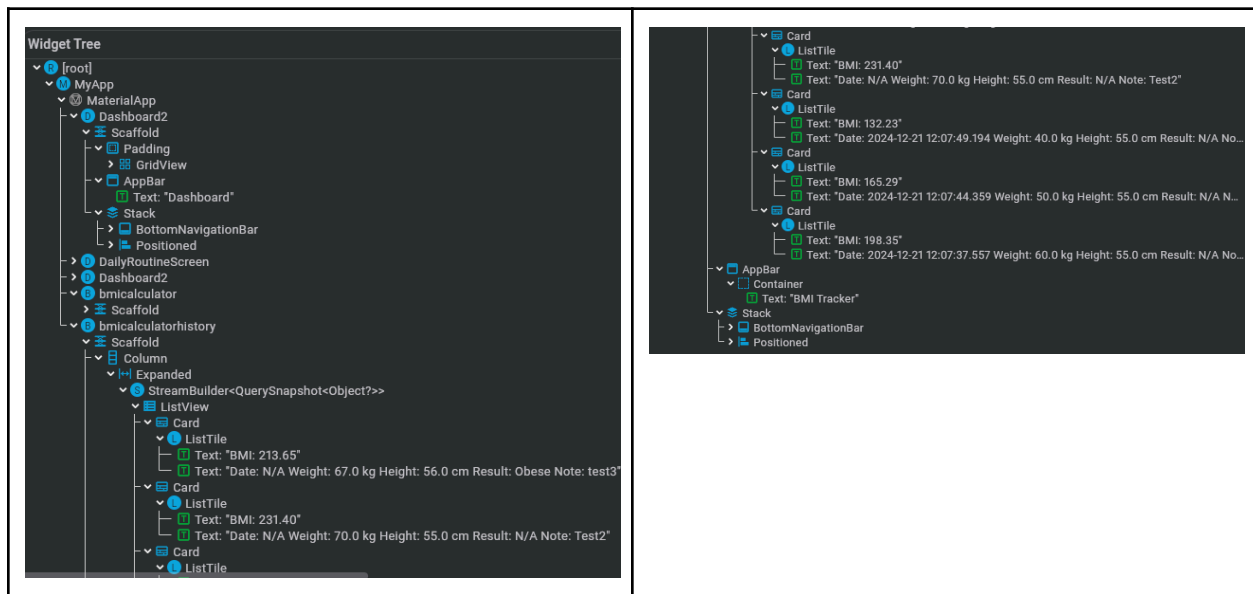
1.2.2 Custom Workout Widget Tree Provided by the Flutter Tools



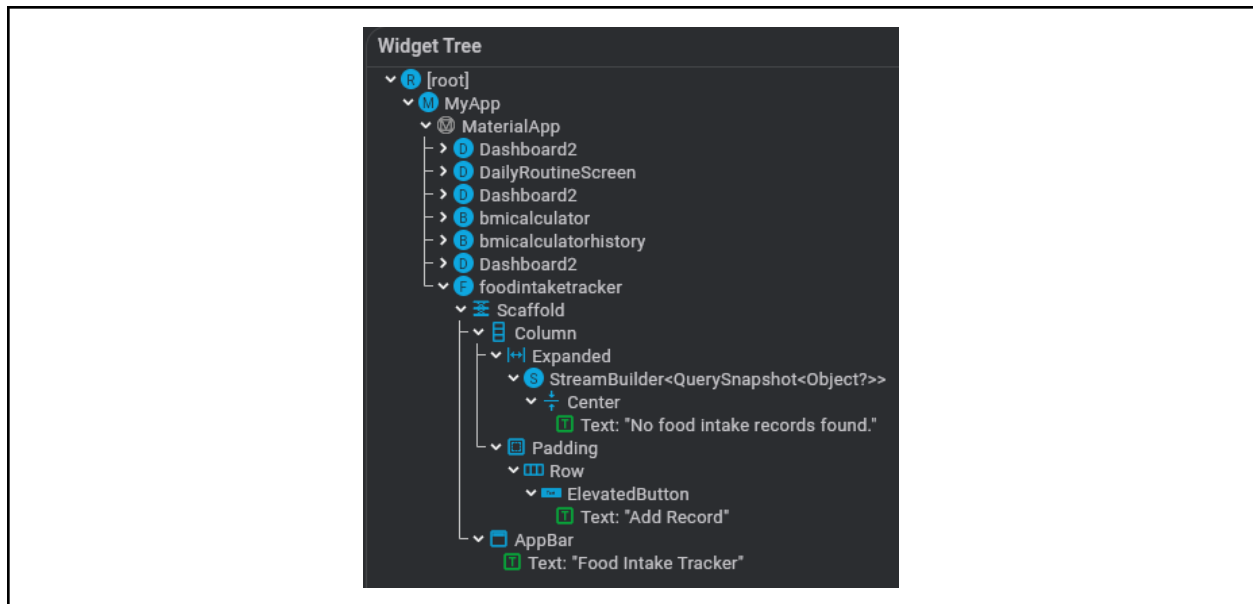
1.3 Workout Track Widget Tree Provided by the Flutter Tools



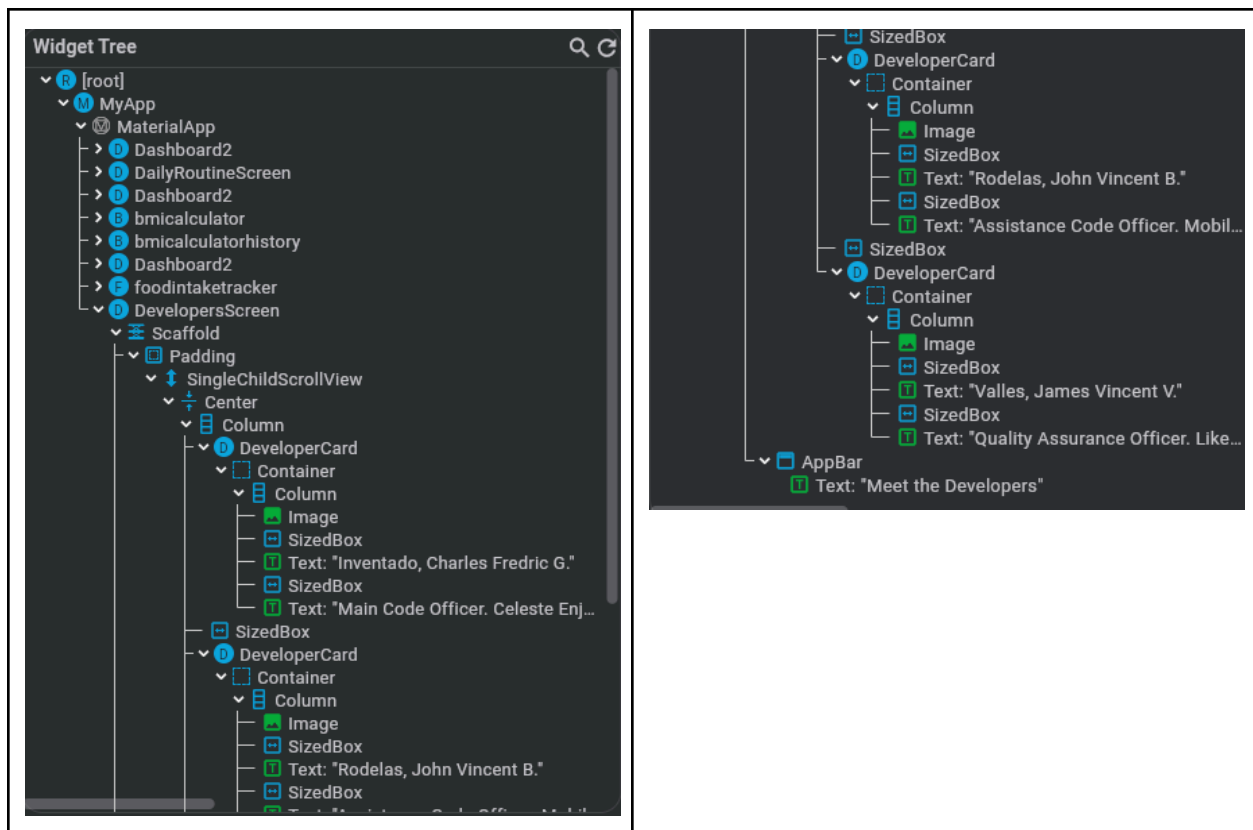
1.4 BMI Calculator Widget Tree Provided by the Flutter Tools



1.5 BMI Track Widget Tree Provided by the Flutter Tools



1.6 Food Intake Widget Tree Provided by the Flutter Tools



1.7 Developers Widget Tree Provided by the Flutter Tools

Flutter Project Code

Sample Code #1 - Timer

```
void _startTimer() {
  setState(() {
    _isTimerActive = true;
    _timerSeconds = 30;
  });
  _timer?.cancel();
  _timer = Timer.periodic(const Duration(seconds: 1), (timer) {
    setState(() {
      if (_timerSeconds > 0) {
        _timerSeconds--;
      } else {
        _isTimerActive = false;
        timer.cancel();
      }
    });
  });
}
```

The `_startTimer()` function initiates a 30-second countdown timer. It updates the UI every second to display the remaining time. Once the timer reaches 0, it stops and updates the UI to reflect its inactive state.

Sample Code #2 - Show Workout Completion Dialog

```
void _showCompletionDialog() {
  showDialog(
    context: context,
    barrierDismissible: false,
    builder: (context) => AlertDialog(
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      title: Text(
        'Workout Complete!',
        style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold, color:
Colors.green[800]),
      ),
      content: Text(
        'Congratulations on completing the ${widget.category} workout!',
        textAlign: TextAlign.center,
        style: const TextStyle(fontSize: 18),
      ),
      actions: [
        TextButton(
          onPressed: () => {
            Navigator.of(context).push(
              MaterialPageRoute(builder: (context) => const Dashboard2()),
            ),
          },
          child: const Text('Go Home', style: TextStyle(color: Colors.green)),
        ),
      ],
    ),
  );
}
```

```
],
),
);
```

This function, `_showCompletionDialog`, displays a dialog announcing the completion of a workout. It features a congratulatory message and a "Go Home" button to navigate back to the dashboard.

Sample Code #3 - Navigation

```
void navigateToCategory(BuildContext context, String category, List<String>
exercises) {
  Navigator.of(context).push(
    MaterialPageRoute(
      builder: (context) => ExerciseCategoryScreen(
        category: category,
        exercises: exercises,
      ),
    ),
  );
}
```

The `navigateToCategory` function navigates to a new screen called `ExerciseCategoryScreen`, passing the category and a list of exercises to it. It uses `MaterialPageRoute` for a platform-specific transition animation.

Sample Code #4 - Confirm Exercise Selection and Navigate

```
@override
void initState() {
  super.initState();
  exerciseCategories.forEach((_, exercises) {
    for (var exercise in exercises) {
      selectedExercises[exercise] = false;
    }
  });
}

// Confirm button to proceed with selected exercises
void _confirmSelection() {
  List<String> chosenExercises = selectedExercises.entries
    .where((entry) => entry.value)
    .map((entry) => entry.key)
    .toList();

  if (chosenExercises.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text("Please select at least one exercise!")),
    );
  } else {
    Navigator.of(context).push(
```

```
MaterialPageRoute(  
  builder: (context) => ConductCustomWorkoutScreen(  
    exercises: chosenExercises,  
  ),  
)  
);  
}
```

This function, `_confirmSelection`, validates the user's exercise selection. If at least one exercise is selected, it navigates to the `ConductCustomWorkoutScreen`. Otherwise, it displays a `snackbar` prompting the user to select an exercise.

Sample Code #5 - User Authentication and Navigation

```
void _submitForm() async {
  if (_formKey.currentState!.validate()) {
    try {
      UserCredential? userCredential;

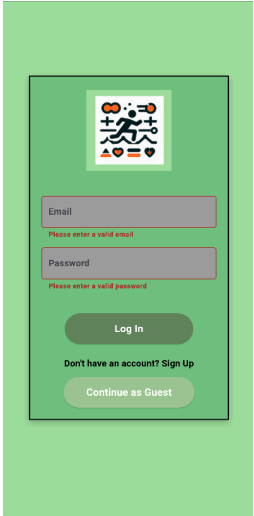
      if (_isLogin) {
        userCredential = await _auth.signInWithEmailAndPassword(
          email: _emailController.text,
          password: _passwordController.text,
        );
      } else {
        userCredential = await _auth.createUserWithEmailAndPassword(
          email: _emailController.text,
          password: _passwordController.text,
        );
      }

      if (userCredential?.user != null) {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => const Dashboard2()),
        );
      } else {
        setState(() {
          _errorMessage = 'Authentication failed. Please try again.';
        });
      }
    } on FirebaseAuthException catch (e) {
      setState(() {
        _errorMessage = 'Error: ${e.message}';
      });
    }
  }
}
```

This function, `_submitForm`, handles user authentication (login or signup) using Firebase. Upon successful authentication, it navigates to the dashboard. If authentication fails, it displays an error message.

App Demo

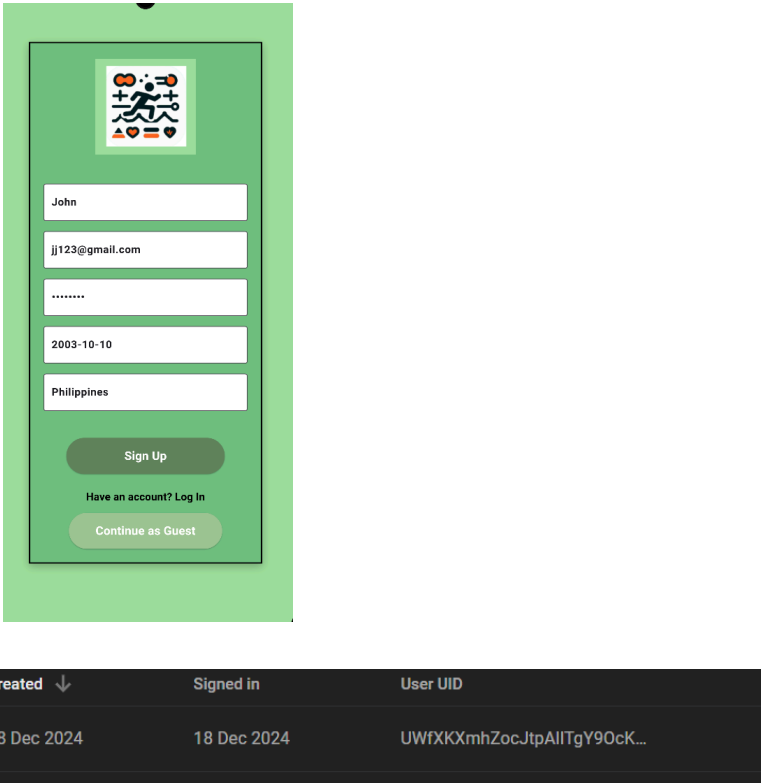
Login - Failure (either validation/error)



The login screen features a green background. At the top is a logo with a running figure and various icons. Below the logo are two input fields: 'Email' and 'Password'. The 'Email' field has a red error message 'Please enter a valid email' below it. The 'Password' field has a red error message 'Please enter a valid password' below it. Below the input fields are three buttons: 'Log In' (dark green), 'Don't have an account? Sign Up' (text link), and 'Continue as Guest' (light green).

2.1.1 User Fail to Login to App

Register

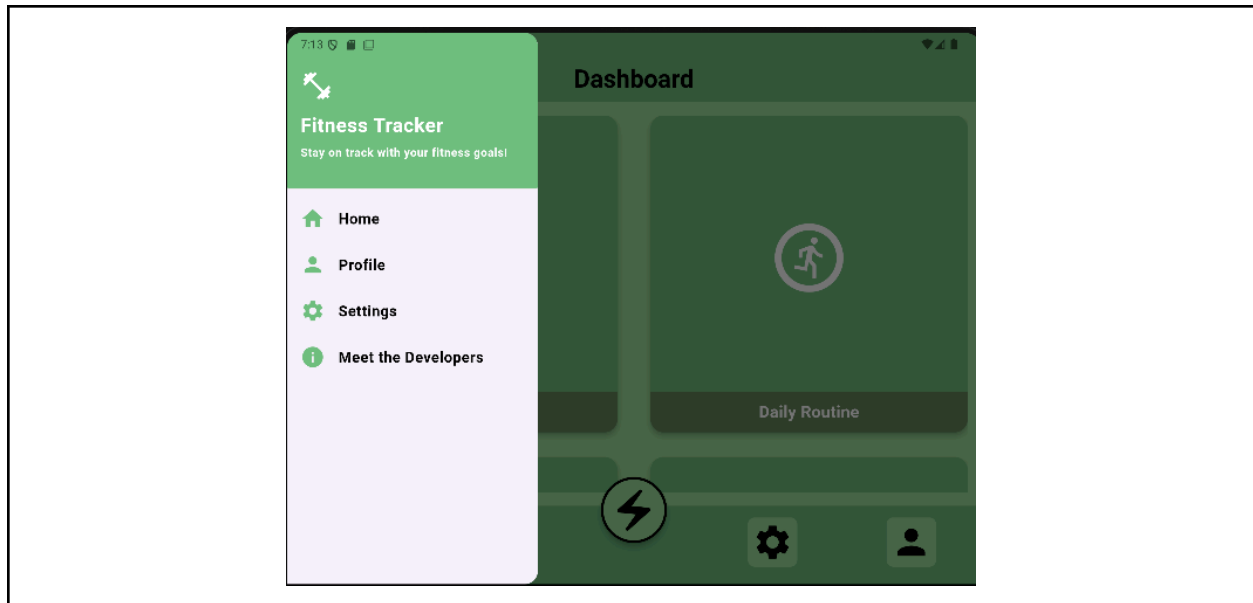


The register screen is on a green background with the same logo. It contains five input fields: 'Name' (filled with 'John'), 'Email' (filled with 'jj123@gmail.com'), 'Password' (filled with '*****'), 'Date of Birth' (filled with '2003-10-10'), and 'Country' (filled with 'Philippines'). Below these are three buttons: 'Sign Up' (dark green), 'Have an account? Log In' (text link), and 'Continue as Guest' (light green).

Identifier	Providers	Created ↓	Signed in	User UID
jj123@gmail.com		18 Dec 2024	18 Dec 2024	UWfXKXmhZocJtpAlITgY90cK...

2.2 User Register to App

Drawer



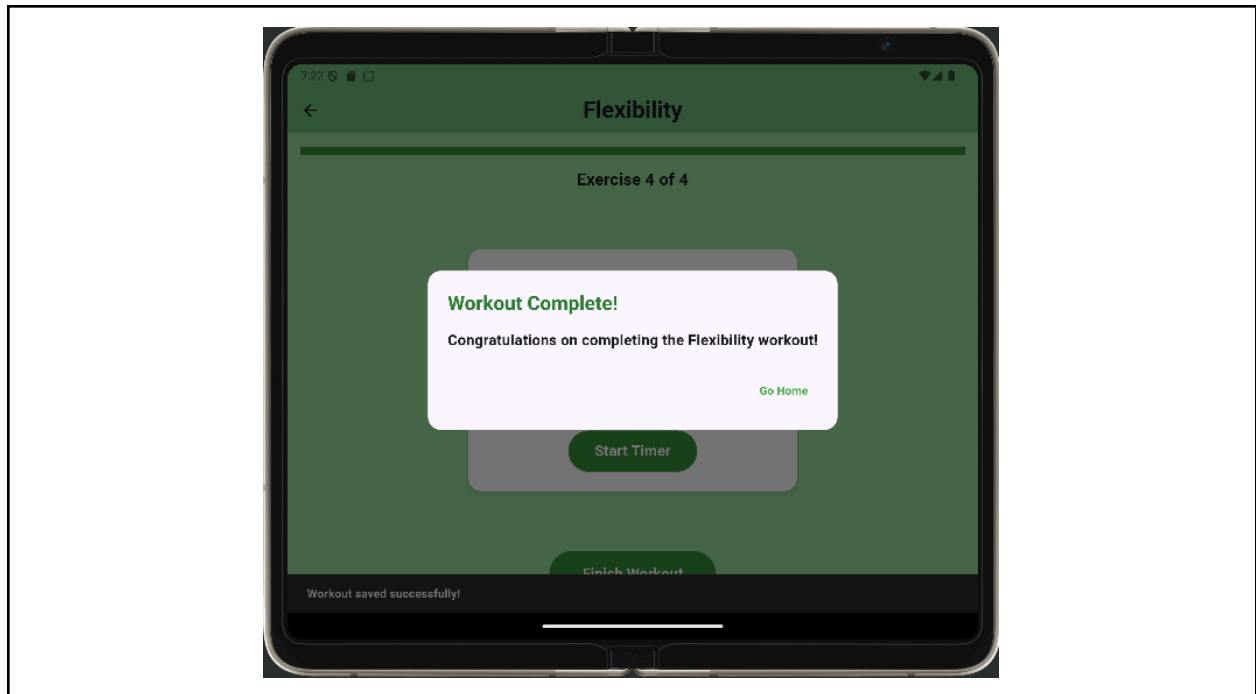
2.3 User Exploring the Drawer on the App

Development Team Screen



2.4 The Development Team Seeing User Activity on the App

Finish Exercise User Did



2.5 App Log the Training User Did

Workout History



2.6 User Views the Training They Did

Recording the Food the User Ate



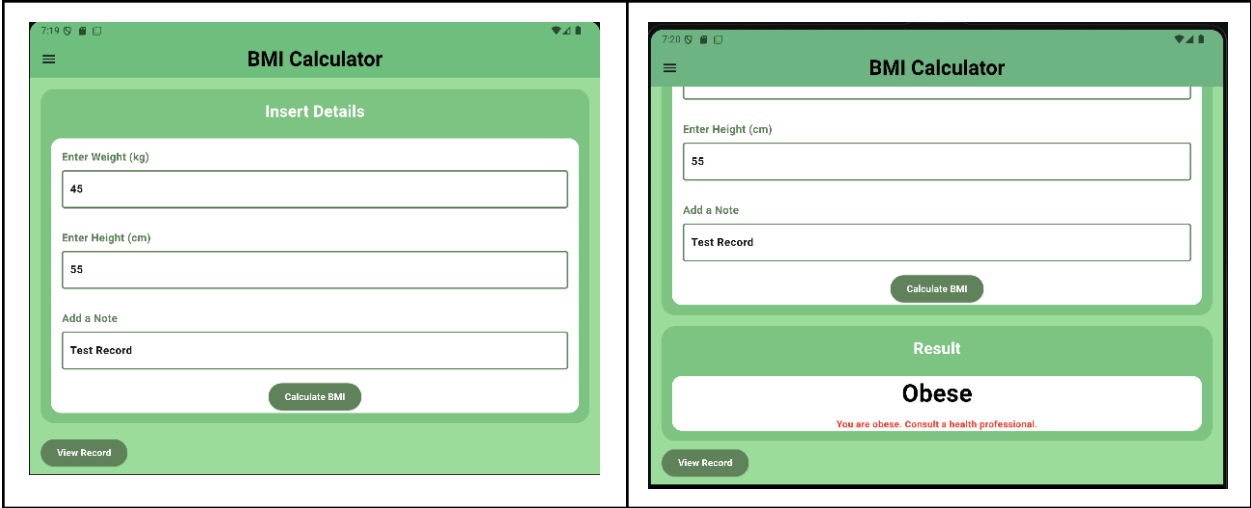
2.7 User Log the Food They Ate

Food Intake History



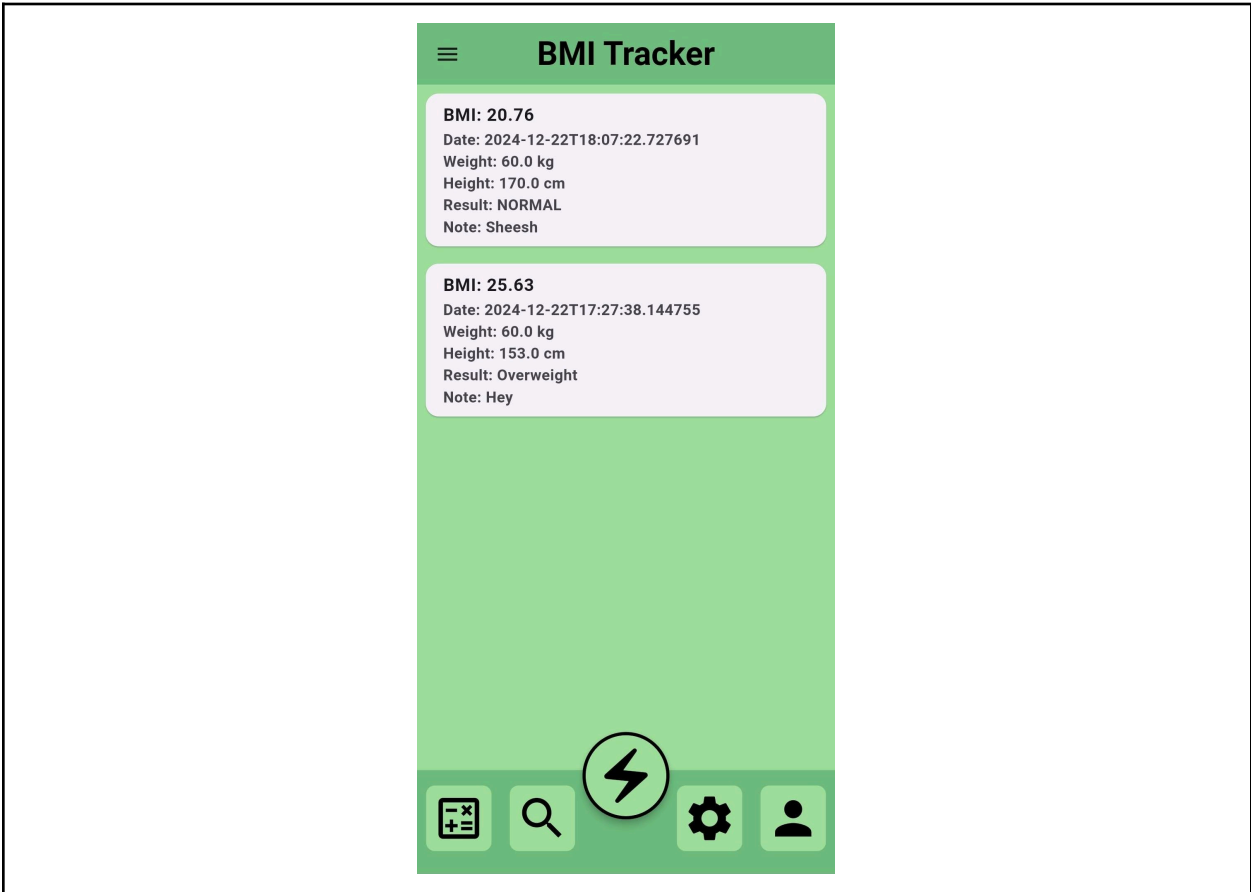
2.8 User Views the Log the Food They Ate

BMI Calculator



2.9 User Calculates BMI

BMI History



2.10 User Calculates BMI